AI BUILD & TEST LAB

Agentic AI QA Automation

Demonstration Evidence Report

Jira + n8n + AI agents for acceptance-criteria analysis, test generation, critic evaluation, synthetic data, mock execution and Jira reporting.

100%	95%	100%	5 / 7	2
AC coverage	Critic score	Final quality	Tests passed	Tests failed

What the Demonstration Proves

Goal

Reduce repetitive test-design effort while retaining critic-agent and human-review controls.

- Extracts acceptance criteria from a sample Jira story
- Generates positive, negative, boundary and security tests
- Evaluates coverage and quality with a critic agent
- Improves test cases when the threshold is not met
- Generates synthetic test data and executes mock API tests
- Summarises failures and prepares potential-defect evidence
- Publishes the consolidated test report to Jira

DEMONSTRATION STORY

Parcel Tracking API Validation

ACCEPTANCE CRITERIA

5 criteria / 100% covered

GENERATED TEST SUITE

7 test cases

EXECUTION

5 passed / 2 failed

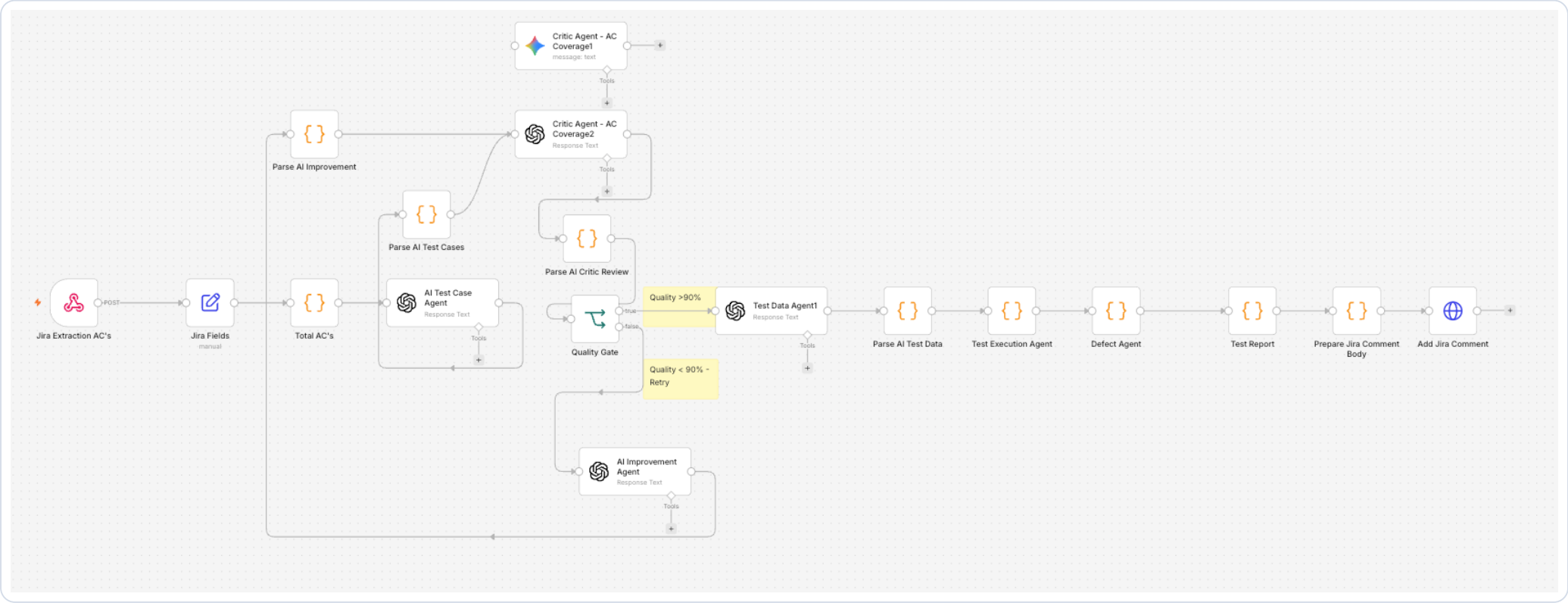
QUALITY PROGRESSION

95% critic review -> 100% final

Interpretation: The latest screenshots represent one consistent seven-test run. These figures replace the earlier draft README values of five tests, four passed and one failed.

Agentic AI QA Workflow

A controlled loop combines generation, evaluation, improvement, execution and reporting.



Control: Critic review evaluates acceptance-criteria coverage and test quality. Low-quality outputs are returned to the improvement agent, with a maximum iteration limit to prevent uncontrolled looping.

Critic Agent: Coverage and Quality Review

ACCEPTANCE CRITERIA
5

TEST CASES REVIEWED
7

COVERAGE
100%

CRITIC SCORE
95%

CRITIC STATUS
PASS

The critic confirmed coverage across positive, negative, boundary and security/privacy scenarios. It noted that a small number of steps were brief but still executable.

OUTPUT Critic output Schema Table JSON

1 item

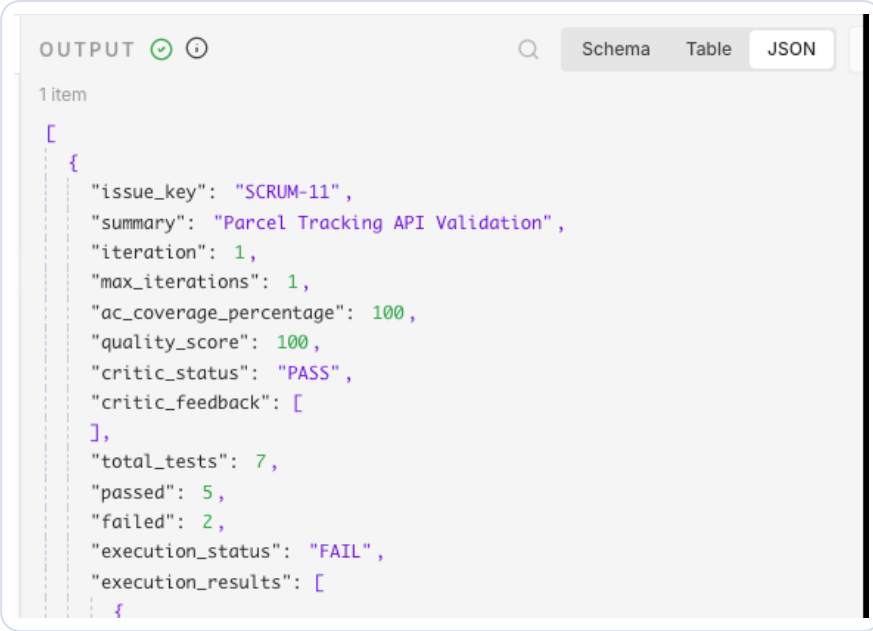

```
[
  {
    "issue_key": "SCRUM-11",
    "summary": "Parcel Tracking API Validation",
    "acceptance_criteria_count": 5,
    "test_case_count": 7,
    "ac_coverage_percentage": 100,
    "quality_score": 95,
    "critic_status": "PASS",
    "critic_feedback": [
      "All acceptance criteria are covered.",
      "Positive, negative, boundary, and security/privacy scenarios are
    included.",
      "Test steps are clear and executable with defined API methods and
    endpoints.",
      "Expected results include HTTP status codes and response validation
    where applicable.",
      "Security/privacy acceptance criterion is well covered with appropriate
    test case.",
      "Minor deduction for a few test steps being slightly brief but still
    executable."
    ],
    "coverage_report": [
      {
        "acceptance_criteria": "1. Customer can track a parcel using a valid
                               tracking number.",
        "covered_by_test_case": "TC_001, TC_006",
        "coverage_status": "Covered"
      },
      {
        "acceptance_criteria": "2. System displays current parcel status,
                               last scan location, and estimated delivery
                               date.",
        "covered_by_test_case": "TC_002",
        "coverage_status": "Covered"
      },
```

Mock API Test Execution

7	5	2
Total tests	Passed	Failed

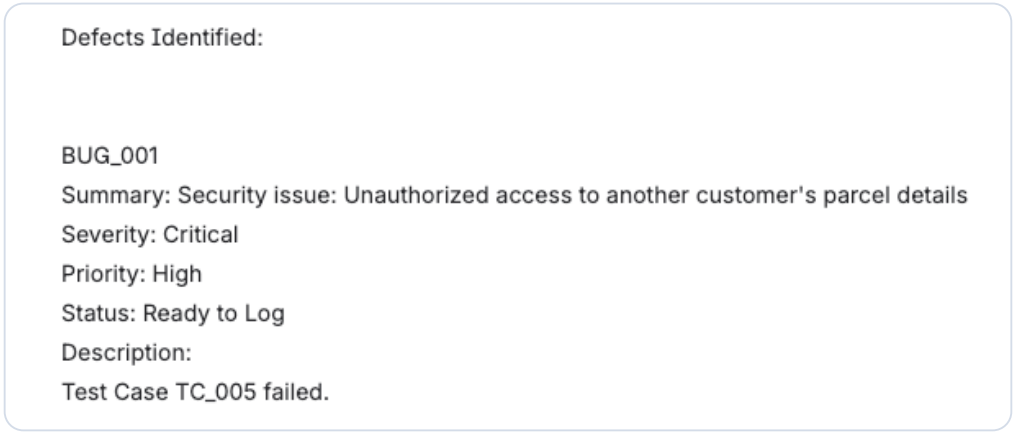
Execution interpretation

- Final acceptance-criteria coverage: 100%
- Final quality score recorded by execution stage: 100%
- Overall status: FAIL because two scenarios failed
- Pass rate: 71.4% (5 of 7)
- Requests used synthetic tracking and customer identifiers
- Execution targeted a local mock service, not production



Screenshot excerpt is cropped to the execution summary. Local mock endpoint details are intentionally excluded.

Security Failure: Access-Control Validation



Candidate ID	BUG_001
Failed test	TC_005
Scenario	Prevent access to another customer's parcel details
Expected status	403 - Forbidden
Actual status	200 - OK
AI-assigned severity	Critical
AI-assigned priority	High

QA interpretation

The system returned parcel information when the test expected access to be denied. This is credible security-test evidence, but it remains an AI-generated potential defect until a QA professional reproduces it, confirms the requirement and rules out test-data or mock-service configuration issues.

Recommended human action: reproduce, verify authorisation rules, inspect request identity and confirm whether a Jira defect should be created.

Portfolio Evidence Summary

Demonstrated capabilities

- Jira-to-n8n workflow orchestration
- Acceptance-criteria analysis and test generation
- Critic-agent evaluation and controlled improvement
- Synthetic data generation and mock API execution
- Pass/fail aggregation and potential-defect analysis
- Jira-ready reporting with human-review controls

Responsible-use boundaries

This report contains demonstration evidence only. All stories, identifiers, test data and endpoints are synthetic or mock. No production credentials, customer records or employer information are included.

AI-generated tests, scores and defect assessments require deterministic checks and professional review before use in a real delivery environment.

Prakash Ganji | Senior QA / Test Lead | AI QA

GitHub: github.com/prakash-aibuildtestlab | YouTube: @AIBuildTestLab